

MINI-TP À REMETTRE

Service Utilisateurs

Fonctionnalités critiques, plan de tests, tableau de synthèse et scénario fonctionnel complet — volets frontend et backend.

Système de billetterie intelligente

ReactJS · Node.js / Express · MongoDB

18 juillet 2026

Mini-TP — Service Utilisateurs

Projet : Système de billetterie intelligente avec QR Code et abonnements. Périmètre : Service Utilisateurs, sur toute la chaîne — interface React et API Node/Express/MongoDB.

Le travail a été réparti en binôme : une personne sur le front (branches `feature/*`, `bugfix/*`, `test/tp1-service-utilisateurs-front`), une sur le back (`refactor/conformite-cahier-des-charges`, `test/service-utilisateurs`). Ce document rassemble les deux côtés parce que le TP demande un seul plan de tests et un seul tableau de synthèse, pas deux documents parallèles qui se recoupent à moitié.

1. Fonctionnalités critiques

On a classé les fonctionnalités selon ce qui se passe si elles tombent en panne : perte de sécurité ou de données d'un côté, simple gêne pour l'admin de l'autre.

Critiques (sécurité ou intégrité des comptes) :

- Authentification JWT — sans elle, plus personne n'entre dans l'application.
- Contrôle d'accès administrateur — une faille ici laisserait un Agent ou un Client gérer tous les comptes.
- Hachage des mots de passe (bcrypt) — un stockage en clair expose tout en cas de fuite de la base.
- Activation de compte + mot de passe temporaire — c'est ce qui rend un compte réellement utilisable après création ou import CSV.
- Modification d'un compte, notamment le rôle — un changement de rôle mal contrôlé équivaut à une élévation de privilèges.
- Redirection forcée tant que le mot de passe temporaire n'est pas changé — sans ce verrou, un mot de passe temporaire resterait valide indéfiniment.

Importantes (effet de masse ou irréversible, mais pas une faille de sécurité en soi) :

- Import CSV — une erreur de traitement se répercute sur des dizaines de comptes d'un coup.
- Actions groupées (activer/bloquer/supprimer plusieurs comptes) — même logique, effet multiplié.
- Suppression — volontairement une suppression logique (`status: 'Supprimé'`), pas un `DELETE` en base, pour ne rien perdre par erreur.
- Client API centralisé côté front (`services/api.js`) — toutes les pages en dépendent, un bug ici se propage partout.

Secondaires (utilité opérationnelle, pas de risque en cas de panne) :

- Recherche et filtrage (rôle, statut, email, téléphone, identifiant).
- Tableau de bord statistique (compteurs total / actifs / bloqués / supprimés par rôle et globalement).

- Upload de la photo de profil.
-

2. Plan de tests

On a gardé deux niveaux, avec un outillage différent de chaque côté parce que ça n'avait pas de sens d'imposer le même framework à un projet React et à une API Express :

- **Back** : `node:test` (le lanceur intégré à Node, pas de dépendance en plus) + `supertest` pour taper sur les routes Express sans ouvrir de port, sur une base MongoDB dédiée au test (`billetterie_test_<pid>` , une par processus pour éviter les interférences). L'app Express est extraite dans `src/app.js` , importable directement par les tests sans passer par `server.js` . L'envoi d'e-mail est neutralisé pendant les tests.
- **Front** : Jest, sur les règles de validation de formulaire (`utils/validators.js`) — c'est la seule logique métier du front qui a du sens à tester unitairement sans monter des composants React entiers ni simuler un DOM complet.

Le reste (rendu des pages, interactions utilisateur en pixels) est vérifié manuellement via le scénario du §4, pas par des tests automatisés — ça aurait demandé React Testing Library + jsdom pour un gain limité vu la taille du projet.

Commandes :

```
# backend
npm test           # toute la suite
npm run test:unitaires
npm run test:api

# frontend
npm test           # 22 tests Jest sur validators.js
```

Hors périmètre pour ce TP : service Abonnements, service Billetterie/QR Code — pas encore développés, on reste sur la gestion des utilisateurs comme demandé.

3. Tableau de synthèse

Back — 59 tests (14 unitaires, 45 API), tous verts

ID	Cas de test	Type	Résultat attendu
U1-U3	Mot de passe temporaire : 8 caractères, alphanumérique, différent à chaque appel	Unitaire	Format et unicité respectés
U4-U7	Échappement des caractères spéciaux dans la recherche (<code>escapeRegex</code>)	Unitaire	Un <code>+</code> ou un métacaractère ne casse plus la requête
U8-U9	Sérialisation du modèle <code>User</code>	Unitaire	Mot de passe jamais exposé, <code>id</code> présent
U10-U14	Validation du modèle (email en minuscule, valeurs par défaut, champs requis, énumérations)	Unitaire	Erreurs de validation levées correctement
A1-A8	Authentification et contrôle d'accès	API	200/401/403 selon les cas
A9-A14	Création de compte, doublons, recherche par email/téléphone/identifiant	API	Codes et résultats attendus, y compris la recherche par téléphone <code>+221...</code>
A15-A19	Filtres rôle/statut, statistiques	API	Résultats correctement filtrés et agrégés
A20-A27	Activation, blocage, suppression logique, actions groupées	API	Statuts appliqués, mots de passe régénérés seulement à l'activation
A28	Verrouillage tant que le mot de passe temporaire n'est pas changé	API	403 puis 200 après changement
A29-A34	Import CSV (valide, doublon, lignes invalides, fichier non-CSV, sans auth)	API	Import partiel correct, erreurs détaillées ligne par ligne
A35-A45	CRUD complet sur un compte (lecture, modification, garde-fous sur email/statut/rôle)	API	Édition possible, mais email et statut restent protégés

Front — 22 tests unitaires (Jest, sur `validators.js`)

Fonction testée	Cas couverts	Nb
<code>isValidEmail</code>	email valide, sans <code>@</code> , valeur vide	3
<code>isValidPhone</code>	9 chiffres seuls, préfixé <code>+221</code> , préfixé <code>221</code> , trop court, trop long, caractères non numériques, vide	7
<code>validateUserForm</code>	formulaire valide, champ requis manquant, email invalide, téléphone invalide, email facultatif en édition, aucun champ fourni	6
<code>validateLoginForm</code>	email vide, mot de passe trop court, cas valide	3
<code>validateNewPassword</code>	mot de passe trop court, confirmation différente, cas valide	3

Ces règles étaient à l'origine écrites directement dans `Login.jsx` , `CreateUserModal.jsx` et `ProfileSettings.jsx` , donc impossibles à tester sans monter tout le composant. On les a sorties dans `utils/validators.js` sans changer le comportement.

4. Scénario fonctionnel complet

De l'import CSV d'un lot d'agents jusqu'à leur première connexion, écran par écran.

Préalable : un administrateur actif existe (`npm run seed:admin`).

1. L'administrateur se connecte sur `/login` . Un jeton JWT est délivré, il arrive sur le Dashboard.
2. Il ouvre **Gestion des Comptes** → **Importer CSV**, dépose un fichier de 10 agents. L'import réussit, les 10 comptes apparaissent avec le statut `Bloqué` . Le fichier contenait volontairement un doublon et deux lignes invalides : la modale affiche le détail ligne par ligne (numéro de ligne, email, raison du rejet), l'import continue pour le reste.
3. Il filtre la liste sur le rôle `Agent` , puis recherche un agent par son numéro `+221...` — la recherche fonctionne (c'est justement le bug qu'on a corrigé, voir §5.1).
4. Il sélectionne 3 agents et clique sur **Activer** dans la barre d'actions groupées. Leur statut passe à `Actif` , un mot de passe temporaire de 8 caractères est généré pour chacun et envoyé par e-mail. Les compteurs de la carte Agents (Actifs/Bloqués) se mettent à jour immédiatement.
5. Un des agents reçoit son mot de passe temporaire et se connecte sur `/login` .
6. Comme `mustChangePassword` vaut `true` , il est redirigé automatiquement vers `/profile` et ne peut aller nulle part ailleurs — le formulaire d'informations personnelles reste désactivé tant qu'il n'a pas changé son mot de passe. Une tentative d'accès direct à une autre route renvoie 403 côté API.

7. Il saisit son ancien mot de passe (temporaire) et un nouveau de 8 caractères minimum, confirmé à l'identique. Le changement est accepté, le bandeau d'avertissement disparaît, il peut maintenant modifier ses informations et sa photo de profil.
8. Retour côté administrateur : il modifie le numéro de téléphone d'un des agents directement depuis le tableau (édition CRUD), le changement est reflété immédiatement.
9. Il bloque puis supprime un autre compte (suppression logique — le statut passe à **Supprimé**, l'enregistrement reste en base).
10. Il consulte le tableau de bord : les compteurs total / actifs / bloqués / supprimés, par rôle et globalement, reflètent exactement les actions effectuées.

Règle métier vérifiée du début à la fin : un compte n'est jamais utilisable avant activation explicite, et un mot de passe temporaire ne donne accès à rien tant qu'il n'a pas été remplacé.

5. Anomalies trouvées pendant le TP

Écrire les tests et refaire le tour de l'app a fait remonter quatre problèmes réels, pas juste théoriques.

5.1 — Recherche par téléphone : erreur 500. Le filtre construisait une regex directement à partir de la saisie (`new RegExp(search.trim(), 'i')`). Comme tous les numéros du projet commencent par **+221**, et qu'un **+** en tête de motif est un quantificateur invalide en regex, toute recherche par téléphone plantait côté serveur. Corrigé en échappant la saisie avant de construire la regex — ça règle aussi un risque d'injection de regex au passage.

5.2 — Champ de fichier CSV hors de sa zone de dépôt. Le `input[type=file]` de la modale d'import était positionné en absolu sans ancrage : il se calait sur toute la fenêtre modale, invisible, et interceptait les clics destinés aux boutons. Impossible de déclencher l'import. Corrigé en ancrant le champ à sa zone de dépôt.

5.3 — Rôle invalide : erreur 500 au lieu de 400. En modifiant un compte avec un rôle hors énumération, l'erreur remontait telle quelle depuis Mongoose en 500. Une saisie invalide, ça reste une erreur du client (400), pas une panne serveur. Corrigé par une validation explicite du rôle avant l'enregistrement.

5.4 — Compteurs "Supprimés" invisibles côté interface. Le cahier des charges demande explicitement 4 compteurs par carte (total, actifs, bloqués, supprimés), et le back les renvoyait bien — mais aucune des 4 cartes de statistiques ne les affichait à l'écran, seuls Total/Actifs/Bloqués étaient visibles. Corrigé côté front.

Le point commun entre 5.1 et 5.4 : les deux étaient masqués par l'interface elle-même (le filtre front recalcule ses propres stats en repli, et personne ne tape "+221" à la main en testant à l'œil). Ce sont typiquement les bugs qu'un test automatisé attrape et qu'un clic rapide dans l'app rate.

6. Limites connues

- L'envoi d'e-mail n'est pas couvert par les tests automatiques (neutralisé pendant les tests pour ne rien envoyer réellement) ; il a été vérifié manuellement une fois.
 - Pas de test de bout en bout automatisé (Cypress/Playwright) reliant vraiment le front au back — le scénario du §4 a été rejoué à la main.
 - Les tests back demandent une instance MongoDB locale ; passer à `mongodb-memory-server` supprimerait cette dépendance.
 - Services Abonnements et Billetterie/QR Code non développés à ce stade, conformément au périmètre imposé pour ce TP.
-

7. Traçabilité

Élément	Emplacement
Tests unitaires back	<code>backend/tests/unitaires/</code>
Tests API back	<code>backend/tests/api/</code>
Utilitaires de test back	<code>backend/tests/helpers.js</code>
Application Express testable	<code>backend/src/app.js</code>
Tests unitaires front	<code>frontend/src/utils/validators.test.js</code>
Règles de validation front	<code>frontend/src/utils/validators.js</code>
Jeux de données CSV	<code>docs/exemples/</code>
